
TP 2.1: GENERADORES PSEUDOALEATORIOS

Berruhet, Marcos
47120
marcos.berruhet@gmail.com

Ceschan, Franco
42737
francoceschan@gmail.com

Nicolas, Pedro
51857
peedrooo1234@gmail.com

Spitale, Camila
50429
camilaspitale26@gmail.com

5 de junio de 2026

ABSTRACT

El presente trabajo tiene como objetivo analizar el comportamiento de distintos generadores de números pseudoaleatorios implementados en Python. Para ello, se desarrollaron generadores propios, como el Generador Congruencial Lineal y el método de los cuadrados medios, y se los comparó con el generador provisto por Python, basado en Mersenne Twister. A partir de una muestra de 10.000 valores para cada generador, se aplicaron medidas descriptivas y pruebas estadísticas de uniformidad e independencia. Además, se generaron histogramas y gráficos de dispersión para complementar el análisis numérico con una interpretación visual de las secuencias obtenidas.

Keywords Simulación · números pseudoaleatorios · Generador Congruencial Lineal · cuadrados medios · Mersenne Twister · pruebas estadísticas

1. Introducción

En el campo de la simulación, los números pseudoaleatorios cumplen un rol fundamental, ya que permiten representar comportamientos inciertos dentro de modelos computacionales. A diferencia de los números verdaderamente aleatorios, los números pseudoaleatorios son generados mediante algoritmos determinísticos. Esto significa que, dada una misma semilla inicial y un mismo procedimiento de generación, la secuencia obtenida será siempre la misma.

Esta característica resulta útil desde el punto de vista experimental, ya que permite repetir simulaciones bajo las mismas condiciones iniciales. Sin embargo, también implica que la calidad del generador debe ser evaluada, dado que una secuencia mal generada puede introducir patrones, sesgos o dependencias que afecten los resultados de una simulación.

En este trabajo se implementan y comparan tres generadores:

- Generador Congruencial Lineal.
- Método de los cuadrados medios.
- Generador de Python basado en Mersenne Twister.

El análisis se realiza sobre 10.000 valores generados por cada método. Para evaluar el comportamiento de las secuencias, se comparan la media y la varianza muestral con sus valores teóricos, y se aplican pruebas estadísticas de uniformidad e independencia. Además, se utilizan histogramas y gráficos de dispersión para observar visualmente posibles patrones, concentraciones o irregularidades.

2. Objetivos

El objetivo general del trabajo es implementar y analizar generadores de números pseudoaleatorios en Python.

Los objetivos específicos son:

- Implementar dos generadores pseudoaleatorios.
- Comparar los generadores implementados con el generador provisto por Python.
- Generar muestras de tamaño suficiente para analizar el comportamiento de cada método.
- Representar gráficamente las secuencias obtenidas mediante histogramas y gráficos de dispersión.
- Aplicar pruebas estadísticas que permitan evaluar uniformidad e independencia.
- Elaborar conclusiones sobre el comportamiento observado y esperado de cada generador.

3. Marco teórico

3.1. Números pseudoaleatorios

Un número pseudoaleatorio es un valor generado por un algoritmo que intenta imitar el comportamiento de una variable aleatoria. Si bien la secuencia puede parecer aleatoria desde un punto de vista estadístico, en realidad surge de una regla matemática determinística.

En general, un generador pseudoaleatorio parte de una semilla inicial. A partir de ella, el algoritmo produce una secuencia de valores:

$$x_0, x_1, x_2, \dots, x_n$$

Luego, estos valores suelen normalizarse para obtener números dentro del intervalo:

$$0 \leq u_i < 1$$

donde cada u_i representa un número pseudoaleatorio uniforme en el intervalo $[0, 1)$.

3.2. Generador Congruencial Lineal

El Generador Congruencial Lineal, también conocido como GCL, es uno de los métodos clásicos para generar números pseudoaleatorios. Su ecuación general es:

$$X_{n+1} = (aX_n + c) \bmod m$$

donde:

- X_n es el estado actual del generador.
- a es el multiplicador.
- c es el incremento.
- m es el módulo.
- X_0 es la semilla inicial.

Para obtener valores en el intervalo $[0, 1)$, se normaliza cada estado dividiéndolo por el módulo:

$$U_n = \frac{X_n}{m}$$

En el código implementado se utilizaron los siguientes parámetros:

$$a = 12345$$

$$c = 0$$

$$m = 2^{31} - 1$$

$$X_0 = 12345$$

Al tener incremento cero, el generador utilizado corresponde a una variante multiplicativa del generador congruencial lineal. El desempeño de este método depende fuertemente de la elección de sus parámetros, ya que una selección inadecuada puede generar ciclos cortos o patrones no deseados.

Para un GCL con $c \neq 0$, el teorema de Hull-Dobell establece condiciones suficientes para que la secuencia alcance el período máximo m : que c y m sean primos entre sí, que $a - 1$ sea divisible por todos los factores primos de m , y que, si m es múltiplo de 4, entonces $a - 1$ también lo sea. En la variante multiplicativa ($c = 0$), se suele exigir que m sea primo, que a sea raíz primitiva módulo m y que la semilla X_0 no sea cero.

Elegir bien los parámetros significa maximizar el período y reducir patrones detectables en las pruebas; elegir mal implica ciclos cortos, repetición temprana de valores o sesgos en la distribución.

En el ejemplo implementado, $m = 2^{31} - 1$ es primo y $c = 0$, por lo que corresponde a la variante multiplicativa. El multiplicador $a = 12345$ resulta raíz primitiva módulo m , condición que garantiza período máximo $m - 1$ cuando la semilla es distinta de cero. La semilla $X_0 = 12345$ cumple únicamente ese requisito ($X_0 \neq 0$): no surge de una regla específica del teorema de Hull-Dobell, que en este caso no aplica directamente porque $c = 0$. Se adoptó el mismo valor que a por simplicidad y reproducibilidad, pero cualquier semilla no nula en el rango válido habría producido un ciclo de la misma longitud, comenzando en un punto distinto de la secuencia.

Es decir, la elección de a y m sí puede evaluarse con el marco teórico de período máximo; la semilla, en cambio, se fijó de forma práctica. En este trabajo no se demostró teóricamente la optimalidad global de los parámetros adoptados, sino que se evaluó su comportamiento de forma empírica mediante las pruebas estadísticas aplicadas a la muestra generada.

3.3. Método de los cuadrados medios

El método de los cuadrados medios consiste en elevar al cuadrado la semilla actual y luego tomar los dígitos centrales del resultado para construir la nueva semilla. El procedimiento general es:

1. Se parte de una semilla inicial de d dígitos.
2. Se eleva la semilla al cuadrado.
3. Se completa el resultado con ceros a la izquierda si es necesario.
4. Se extraen los d dígitos centrales.
5. Esos dígitos pasan a ser la nueva semilla.

En este trabajo se utilizó una semilla de cuatro dígitos:

$$X_0 = 9731$$

y se trabajó con:

$$d = 4$$

Por lo tanto, cada valor generado se normaliza mediante:

$$U_n = \frac{X_n}{10^d}$$

Este método es históricamente relevante, aunque suele presentar problemas de ciclos cortos, convergencia a cero o repetición temprana de valores. Por ese motivo, resulta interesante compararlo con generadores más robustos.

3.4. Mersenne Twister de Python

Python incorpora un generador pseudoaleatorio basado en Mersenne Twister. En este trabajo se lo utiliza como generador de referencia, debido a que es el método estándar empleado por el módulo `random`.

Para asegurar la reproducibilidad de los resultados, se utilizó una semilla fija:

$$X_0 = 42$$

La ventaja de este enfoque es que permite comparar los generadores propios contra una herramienta ampliamente utilizada en aplicaciones generales de simulación y programación.

3.5. Media y varianza teórica

Para una distribución uniforme continua en el intervalo $[0, 1)$, la media teórica es:

$$E(U) = 0,5$$

y la varianza teórica es:

$$Var(U) = \frac{1}{12} \approx 0,0833$$

Por este motivo, una primera forma de evaluar las secuencias generadas consiste en comparar la media y la varianza muestral de cada generador con estos valores teóricos. Si bien esta comparación no constituye por sí sola una prueba estadística formal de aleatoriedad, permite detectar desvíos generales en la distribución de los valores generados.

4. Metodología

El desarrollo se realizó en Python. Se definieron clases independientes para cada generador, con métodos para obtener el siguiente valor de la secuencia y para generar una muestra completa de tamaño determinado.

La cantidad de valores generados para cada método fue:

$$N = 10000$$

Además, para las pruebas estadísticas se utilizó un nivel de significancia de:

$$\alpha = 0,05$$

Las semillas utilizadas fueron:

Generador	Semilla	Observación
GCL	12345	Generador implementado manualmente
Método de los cuadrados	9731	Generador implementado manualmente
Python random	42	Generador Mersenne Twister

Cuadro 1: Semillas utilizadas para cada generador.

Para cada generador se obtuvo una lista de valores en el intervalo $[0, 1)$. Luego, se calcularon la media muestral, la varianza muestral, el estadístico Chi-cuadrado y el estadístico de corridas. También se generaron histogramas y gráficos de dispersión, con el objetivo de complementar el análisis numérico mediante una evaluación visual.

El código fuente completo se entrega como archivo complementario. En el presente informe se describe la lógica general de implementación, los generadores utilizados, los parámetros definidos, las pruebas aplicadas y los resultados obtenidos.

5. Pruebas estadísticas aplicadas

El análisis gráfico permite una primera aproximación al comportamiento de los generadores, pero no resulta suficiente para concluir sobre la calidad de las secuencias. Por ese motivo, se complementó el estudio con medidas descriptivas y pruebas estadísticas.

En todos los casos se consideró como distribución teórica esperada una uniforme en el intervalo $[0, 1)$. El nivel de significancia utilizado fue:

$$\alpha = 0,05$$

Bajo este criterio, si el valor-p obtenido es mayor que α , no se rechaza la hipótesis nula. En cambio, si el valor-p es menor o igual que α , se rechaza la hipótesis nula y se considera que existe evidencia estadística de desvío respecto del comportamiento esperado.

5.1. Interpretación de ACEPTA H_0 y RECHAZA H_0

En la Tabla 2, las columnas *Uniformidad* e *Independencia* resumen el resultado de cada prueba formal según el valor-p obtenido:

- **ACEPTA H_0** : el valor-p es mayor que $\alpha = 0,05$. No hay evidencia estadística suficiente para afirmar un desvío respecto del comportamiento esperado. En otras palabras, la muestra analizada es *compatible* con la hipótesis planteada.
- **RECHAZA H_0** : el valor-p es menor o igual que α . Existe evidencia estadística de desvío respecto del comportamiento esperado bajo esa prueba.

En la prueba Chi-cuadrado, H_0 afirma que los datos se distribuyen de forma uniforme en $[0, 1)$. En la prueba de corridas, H_0 afirma que no existe dependencia significativa entre valores consecutivos de la secuencia.

Es importante aclarar que aceptar H_0 no demuestra que el generador sea perfectamente aleatorio. Solo indica que, con la muestra y las pruebas utilizadas, no se detectó un desvío significativo. Del mismo modo, rechazar H_0 no implica que el generador sea inutilizable en todo contexto, pero sí señala un problema que debe considerarse al interpretar los resultados.

5.2. Comparación de media

La media muestral se comparó con la media teórica de una distribución uniforme $U(0, 1)$:

$$E(U) = 0,5$$

Una media muy alejada de este valor puede indicar un sesgo en la generación de los números.

5.3. Comparación de varianza

La varianza muestral se comparó con la varianza teórica de una distribución uniforme $U(0, 1)$:

$$Var(U) = \frac{1}{12} \approx 0,0833$$

Una varianza considerablemente menor puede indicar concentración de valores en una zona del intervalo, mientras que una varianza mayor puede mostrar una distribución más dispersa que la esperada.

5.4. Prueba Chi-cuadrado

La prueba Chi-cuadrado permite evaluar si los valores generados se distribuyen uniformemente en el intervalo $[0, 1)$. Para ello, el intervalo se dividió en diez subintervalos de igual amplitud y se compararon las frecuencias observadas con las frecuencias esperadas.

El estadístico se calcula como:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

donde O_i representa la frecuencia observada en el intervalo i , E_i la frecuencia esperada y k la cantidad de intervalos. La hipótesis nula de esta prueba establece que la muestra es compatible con una distribución uniforme.

5.5. Prueba de corridas

La prueba de corridas permite analizar la independencia de la secuencia generada. En este caso, los valores se clasificaron según se encuentren por encima o por debajo de la media muestral. Luego, se contó la cantidad de corridas, entendidas como grupos consecutivos de valores ubicados del mismo lado de la media.

El estadístico utilizado fue:

$$Z = \frac{R - E(R)}{\sqrt{Var(R)}}$$

donde R representa la cantidad de corridas observadas, $E(R)$ la cantidad esperada de corridas y $Var(R)$ su varianza. La hipótesis nula de esta prueba establece que la secuencia no presenta evidencia significativa de dependencia entre valores consecutivos.

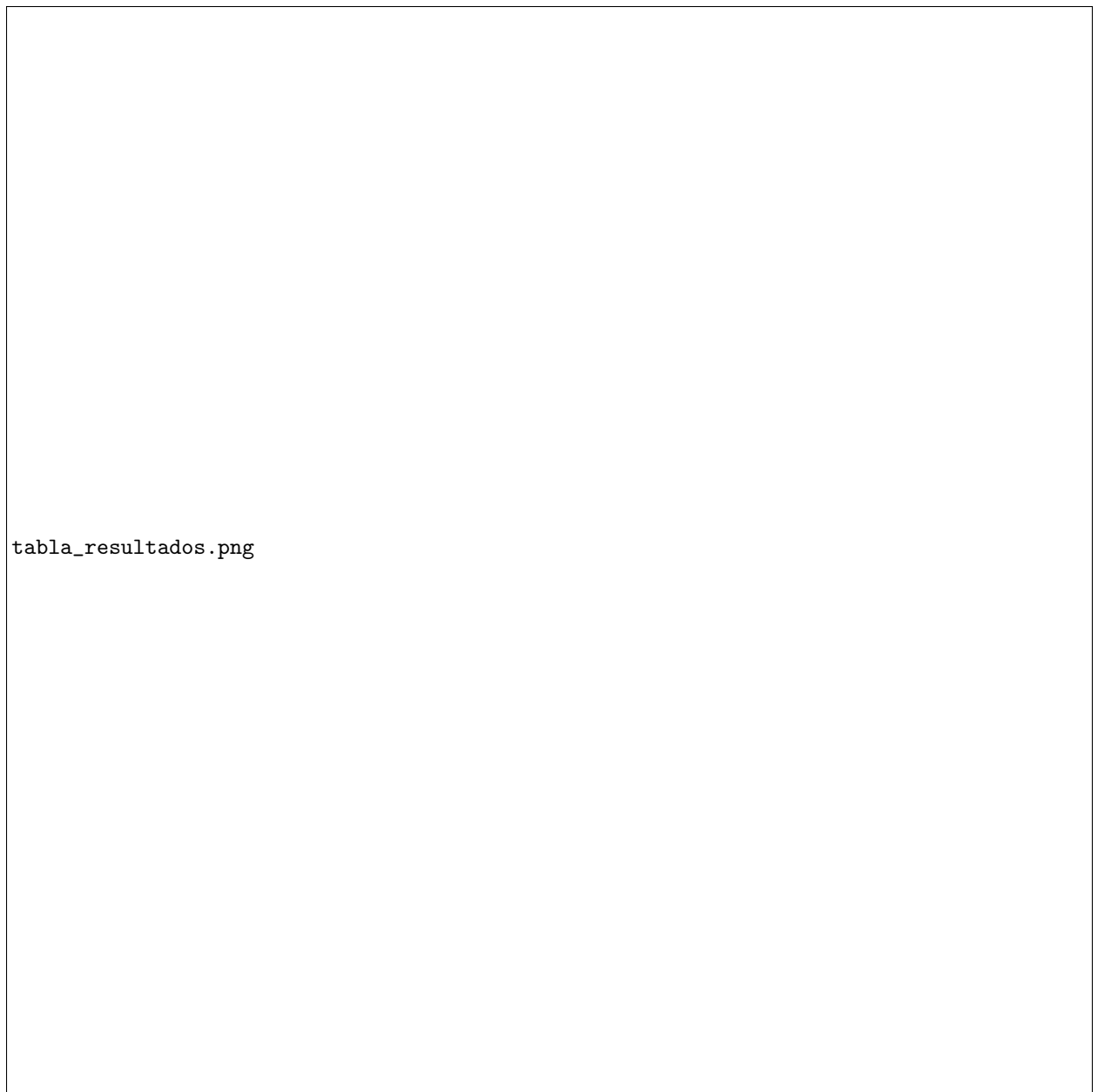
6. Resultados

Los resultados obtenidos se resumen en la Tabla 2. En ella se comparan los valores de media, varianza, pruebas de uniformidad y pruebas de independencia para cada generador analizado.

Generador	Media	Varianza	Chi-cuadrado	p Chi2	Uniformidad	Z corridas	p corridas	Independencia
GCL	0.4943	0.0849	13.3940	0.1456	ACEPTA H0	-1,4360	0.1510	ACEPTA H0
Método cuadrados	0.5098	0.0502	14865.3380	0.0000	RECHAZA H0	-0,0400	0.9681	ACEPTA H0
Python random	0.5003	0.0828	13.6560	0.1351	ACEPTA H0	1.1402	0.2542	ACEPTA H0

Cuadro 2: Resultados estadísticos obtenidos para cada generador.

También se incluye la tabla exportada por el programa, donde se presentan los resultados comparativos de las pruebas aplicadas.



tabla_resultados.png

Figura 1: Tabla comparativa generada a partir de los resultados del programa.

6.1. Resultados del Generador Congruencial Lineal

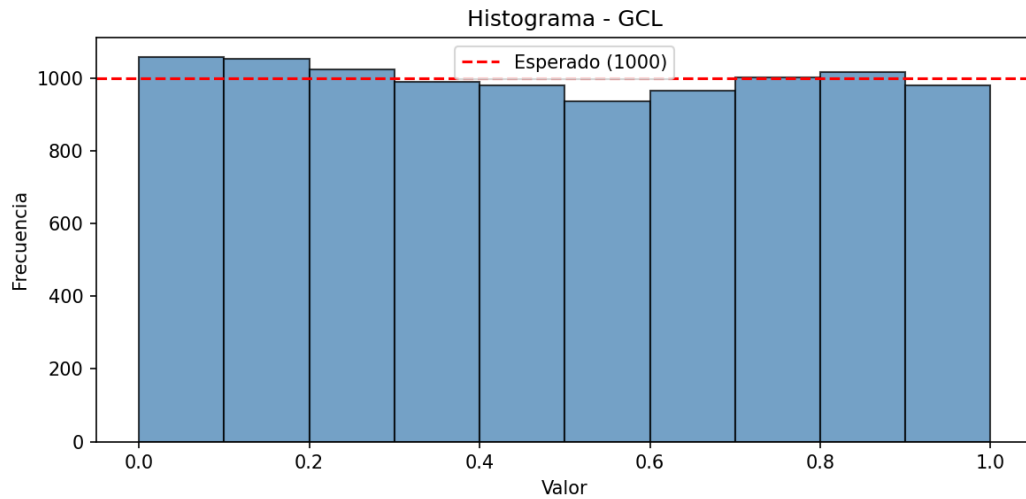


Figura 2: Histograma de valores generados mediante el Generador Congruencial Lineal.

En la Figura 2 se observa la distribución de frecuencias de los valores generados por el GCL. Para una secuencia compatible con una distribución uniforme, se espera que las frecuencias de los intervalos sean similares y se mantengan cercanas a la frecuencia esperada.

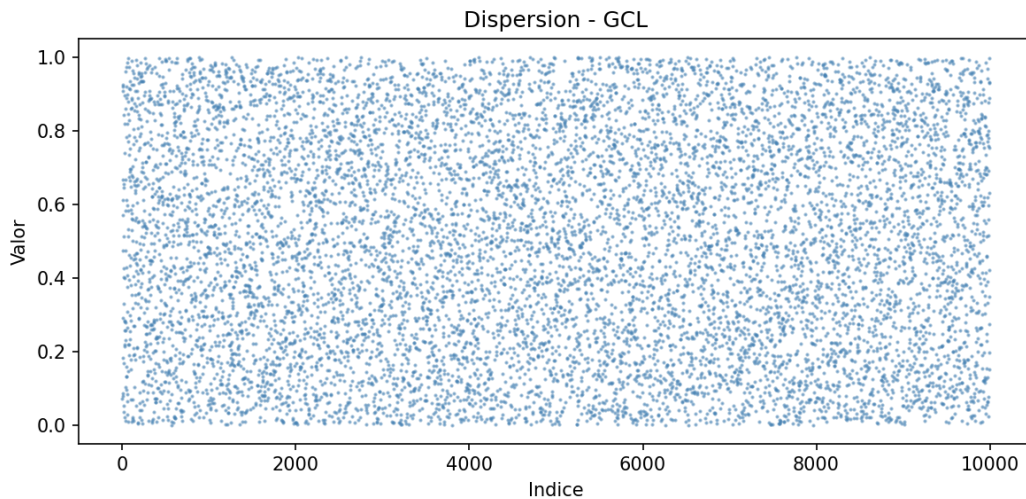


Figura 3: Gráfico de dispersión de valores generados mediante el Generador Congruencial Lineal.

En la Figura 3 se observa la secuencia obtenida mediante el Generador Congruencial Lineal en función del índice de generación. Se espera que los puntos se distribuyan dentro del intervalo $[0, 1)$ sin mostrar una tendencia evidente, ciclos visibles o zonas de concentración sistemática.

6.2. Resultados del método de los cuadrados medios

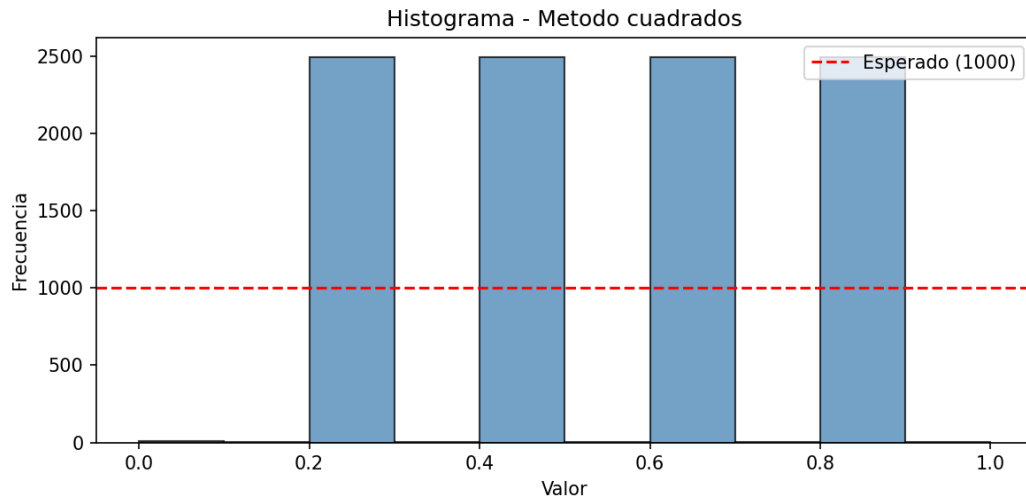


Figura 4: Histograma de valores generados mediante el método de los cuadrados medios.

En la Figura 4 se muestra la distribución de frecuencias obtenida mediante el método de los cuadrados medios. Este método suele presentar problemas de ciclos cortos, repeticiones tempranas o pérdida de variabilidad, por lo que resulta importante observar si las frecuencias se mantienen cercanas al comportamiento uniforme esperado.

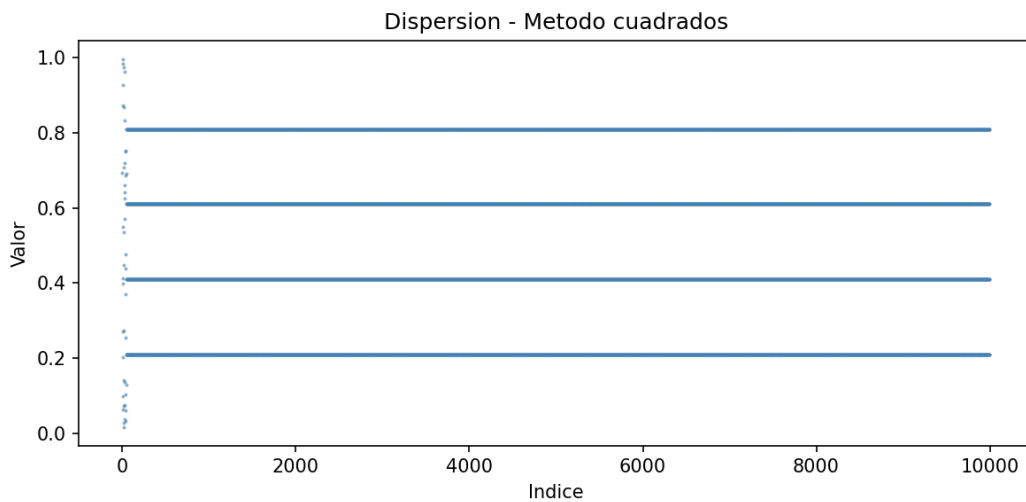


Figura 5: Gráfico de dispersión de valores generados mediante el método de los cuadrados medios.

En la Figura 5 se representa la secuencia generada por el método de los cuadrados medios. La presencia de patrones, valores repetidos o zonas vacías puede indicar limitaciones del método y justificar su menor confiabilidad en comparación con generadores más robustos.

6.3. Resultados del generador de Python

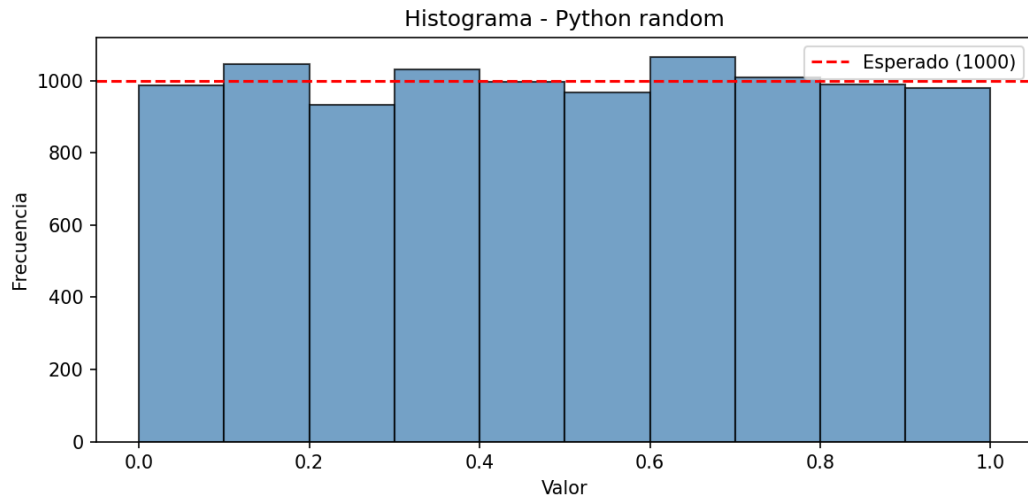


Figura 6: Histograma de valores generados mediante el generador random de Python.

En la Figura 6 se observa la distribución de frecuencias generada por Python. Este generador se utiliza como referencia, por lo que se espera que presente un comportamiento más estable y cercano a una distribución uniforme.

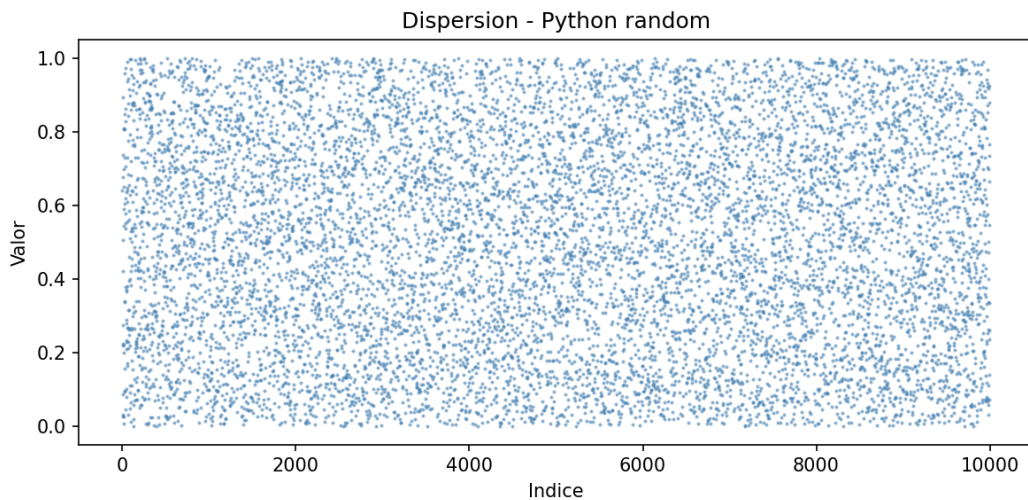


Figura 7: Gráfico de dispersión de valores generados mediante el generador random de Python.

En la Figura 7 se presenta la secuencia generada por Python en función del índice de generación. En un comportamiento adecuado no deberían observarse tendencias marcadas ni estructuras repetitivas evidentes.

7. Análisis de resultados

A partir de las pruebas aplicadas, se puede evaluar el comportamiento de cada generador desde dos perspectivas: la uniformidad de los valores generados y la independencia de la secuencia.

La uniformidad se analiza mediante la comparación de media y varianza, la prueba Chi-cuadrado y los histogramas. Un generador con buen comportamiento debería presentar una media cercana a 0,5, una varianza cercana a 1/12, un valor-p mayor que $\alpha = 0,05$ en la prueba Chi-cuadrado y frecuencias relativamente equilibradas en los histogramas.

La independencia se analiza mediante la prueba de corridas y los gráficos de dispersión. Un generador adecuado debería presentar un valor-p mayor que 0,05 en la prueba de corridas y ausencia de patrones visibles en los gráficos de dispersión.

El Generador Congruencial Lineal es un método simple y eficiente. En este trabajo, con los parámetros adoptados ($a = 12345$, $c = 0$, $m = 2^{31} - 1$, $X_0 = 12345$), la muestra analizada fue compatible con uniformidad e independencia según las pruebas aplicadas. Esto no garantiza que cualquier combinación de parámetros funcione igual: como se indicó en el marco teórico, una mala elección puede producir ciclos cortos o patrones detectables, mientras que una buena elección busca maximizar el período y pasar pruebas estadísticas como las implementadas aquí.

El método de los cuadrados medios es sencillo de implementar, pero suele ser menos confiable. Una de sus principales limitaciones es que puede caer rápidamente en ciclos repetitivos o en valores cercanos a cero. Por lo tanto, aunque tiene importancia histórica, no suele ser recomendable para simulaciones modernas que requieran alta calidad estadística.

El generador de Python, basado en Mersenne Twister, se toma como referencia debido a que es un generador ampliamente utilizado. Se espera que presente un comportamiento más robusto que los métodos simples implementados manualmente.

Es importante aclarar que el hecho de que un generador supere las pruebas aplicadas no demuestra que sea verdaderamente aleatorio. Solo indica que, bajo la muestra analizada y las pruebas utilizadas, no se encontraron evidencias suficientes para rechazar el comportamiento esperado. Del mismo modo, si una prueba rechaza la hipótesis nula, esto no implica necesariamente que el generador sea inutilizable en todos los contextos, pero sí evidencia un desvío que debe ser considerado.

8. Conclusiones

El trabajo permitió implementar y comparar tres generadores de números pseudoaleatorios: el Generador Congruencial Lineal, el método de los cuadrados medios y el generador provisto por Python. Para cada uno se generó una muestra de 10.000 valores, se calcularon medidas descriptivas, se aplicaron pruebas estadísticas y se elaboraron gráficos para analizar su comportamiento.

Desde el punto de vista metodológico, el análisis combinó herramientas numéricas y visuales. La media y la varianza permitieron evaluar una primera aproximación al comportamiento esperado de una distribución uniforme. La prueba Chi-cuadrado permitió analizar la uniformidad de las secuencias. La prueba de corridas permitió estudiar la independencia entre valores generados.

En términos generales, el generador que presente valores cercanos a la media y varianza teórica, valores-p superiores a 0,05 en las pruebas formales y ausencia de patrones visuales puede considerarse compatible con el comportamiento esperado para una secuencia pseudoaleatoria uniforme en $[0, 1)$.

El método de los cuadrados medios resultó el más limitado: con semilla 9731 generó solo 48 valores distintos en 10.000 iteraciones, lo que explica el rechazo de H_0 en uniformidad pese a aceptar independencia en corridas. El Generador Congruencial Lineal, con los parámetros utilizados, mostró un comportamiento compatible con lo esperado en las pruebas formales, lo que confirma la importancia de evaluar empíricamente la calidad de un generador y no basarse solo en su simplicidad de implementación. El generador de Python se utiliza como referencia por tratarse de una implementación más robusta y ampliamente utilizada.

Como conclusión general, la calidad de un generador pseudoaleatorio no debe evaluarse únicamente a partir de una inspección visual. Es necesario combinar gráficos, medidas descriptivas y pruebas estadísticas para obtener una evaluación más completa del comportamiento de las secuencias generadas.

Referencias

- [1] Tereom. *Números pseudoaleatorios*. Disponible en: <https://tereom.github.io/est-computacional-2018/numeros-pseudoaleatorios.html>
- [2] Random.org. *Randomness and Random Numbers*. Disponible en: <https://www.random.org/analysis/>
- [3] Haahr, M. *Random.org: Introduction to Randomness and Random Numbers*. Random.org, 2005. Disponible en: <https://www.random.org/analysis/Analysis2005.pdf>
- [4] Python Software Foundation. *random — Generate pseudo-random numbers*. Disponible en: <https://docs.python.org/3/library/random.html>